

Chapter 9: Lists and Indexing

Gregory S. DeLozier, PhD

Every value this interpreter has handled so far has been a single, immutable thing: a number, a string, a boolean. Even a variable holding one of these can only ever be replaced, never changed from the inside. A **list** is different on both counts: it holds many values at once, in order, and it can be modified in place without being replaced. (1) That second property is new enough to the language that it changes what assignment itself has to mean.

```
scores = [70, 80, 90];
scores[0] = 75;
print(scores);
```

This prints `[75, 80, 90]`. The list `scores` was not replaced; its first element was.

Breaking Changes

Earlier use	Change
A variable, or any identifier, named <code>length</code>	Rename it; it is now a keyword.
A program using <code>[</code> or <code>]</code> for any other purpose	Not previously legal syntax; now reserved for list literals and indexing.

Nothing about numbers, strings, booleans, conditionals, or loops changes. Lists are a new kind of value, built on everything already established, not a replacement for any of it.

List Literals

```
empty = [];
mixed = [1, "two", true];
```

A list literal is a comma-separated sequence of expressions between square brackets, including zero of them. The elements do not need to share a type; `mixed` holds a number, a string, and a boolean at once.

```
if ast["tag"] == "list":
    items = []
    for element in ast["items"]:
        value, status = evaluate(element, environment)
        if status is not None:
            return value, status
        items.append(value)
    return items, None
```

Each element is evaluated in source order, and its status is checked before moving to the next one, the same discipline every other multi-part construct in this interpreter already follows. `[1, exit(7), 3]` never evaluates the 3.

Reading by Index

```
numbers = [10, 20, 30];
print(numbers[1]);
```

This prints 20. An index is a position, counted from 0; it is not the same thing as the element found there, and mixing the two up is the most common mistake a reader of this chapter will make at first. `numbers[1]` is the element at position 1, which happens to be the number 20.

Indexing is **postfix**: `expr[expr]` applies after any expression that produces a list, not only an identifier. `[10, 20, 30][1]` is legal, if unusual, for the same reason `(a + b) * c` is legal: parentheses, and here brackets, attach to whatever came before them.

```
def parse_complex_suffixes(node, tokens):
    # { index_suffix }, applied to a node already parsed. Shared by reading
    # (postfix indexing in an expression) and writing (an assignment
    # target): both are the same chain of "complex" nodes, evaluated
    # differently depending on which side of "=" they end up on. The tag is
    # "complex" rather than "index" on purpose: a later chapter's function
    # calls are the same postfix shape with a different suffix, and will
    # extend this same node instead of introducing a parallel one.
    while tokens[0]["tag"] == "[":
        index, tokens = parse_expression(tokens[1:])
        tokens = require(tokens, "]", "Expected ']' after index")
        node = {"tag": "complex", "base": node, "index": index}
    return node, tokens
```

The evaluator checks status from both the base and the index before touching either one, the same two-child pattern arithmetic has used since the pair of them first appeared:

```
if ast["tag"] == "complex":
    # Reading: base and index are both evaluated and checked for status
    # before the access happens. Assignment to the same node shape is
    # handled separately, under "assign", where this same read logic
    # is what resolves every level except the last.
    base, status = evaluate(ast["base"], environment)
    if status is not None:
        return base, status
    index, status = evaluate(ast["index"], environment)
    if status is not None:
        return index, status
    if type(base) is not list:
        raise TypeError("Indexing requires a list")
    return base[require_list_index(index, base)], None
```

An index must be an integer, or a float with no fractional part: `3.0` is accepted the same as `3`, but `3.5` is not. Negative indices are rejected outright; this chapter never counts backward from the end of a list. Python treats `bool` as a subtype of `int`, so `require_list_index` excludes it by checking the exact type, not just whether the value happens to behave like one:

```
def require_list_index(index, base):
    # An index must be an integer, or a float with no fractional part:
    # 3.0 is accepted, 3.5 is not. Python treats bool as a subtype of int,
    # so it is excluded explicitly rather than by accident. No negative
    # indices, and no silent growth on assignment: both directions of
    # out-of-range access raise the same error.
    if type(index) is bool or type(index) not in (int, float):
        raise TypeError("List index must be an integer")
    if type(index) is float and not index.is_integer():
        raise TypeError("List index must be an integer")
    index = int(index)
    if not (0 <= index < len(base)):
        raise IndexError("List index out of range")
    return index
```

`numbers[true]` and `numbers[1.5]` are both type errors. `numbers[3]`, on a three-element list, is out of range: a clear failure, not three implicitly becoming the list's new length.

Chained Indexing

```
matrix = [[1, 2], [3, 4], [5, 6]];
print(matrix[1][0]);
```

This prints `3`. `matrix[1]` is the list `[3, 4]`; indexing that result with `[0]` reads its first element. There is no special two-argument indexing form for this. `matrix[1][0]` is two "complex" nodes, the outer one using the inner one as its base:

```
{
  "tag": "complex",
  "base": {
    "tag": "complex",
    "base": {"tag": "identifier", "value": "matrix"},
    "index": {"tag": "number", "value": 1},
  },
  "index": {"tag": "number", "value": 0},
}
```

Evaluating the outer node evaluates its base first, which is itself a "complex" node and gets evaluated the same way any expression would be. Nothing about chained indexing needed new evaluator code.

Traversal with While and Length

```
values = [3, 1, 4, 1, 5, 9];
total = 0;
i = 0;
while (i < length(values)) {
    total = total + values[i];
    i = i + 1;
}
```

total ends at 23. length is a dedicated expression, parenthesized like number() or type(), not a general function call; Vertex does not have those yet.

```
if ast["tag"] == "length":
    value, status = evaluate(ast["expression"], environment)
    if status is not None:
        return value, status
    if type(value) is not list:
        raise TypeError("length argument must be a list")
    return len(value), None
```

length only accepts a list. A string's length is a reasonable thing to want, but this chapter does not ask for it, and adding it now would be answering a question nobody asked yet.

Indexed Assignment

```
assignment_statement ::= <identifier> { index_suffix } "=" expression
```

An assignment target is an identifier optionally followed by one or more [index] suffixes, parsed with the exact same `parse_complex_suffixes` helper that builds a read expression. The difference between reading and writing is not in the grammar; it is in what the evaluator does with the resulting node depending on which side of `=` it ends up on.

```
if ast["tag"] == "assign":
    # The target's own subexpressions, if it has any, are resolved before
    # the right-hand value. For an identifier target that resolves
    # nothing (just a name), so this is the same order as before. For an
    # indexed target, the base and the index are both evaluated, and
    # checked for status, ahead of the value.
    target = ast["target"]
    if target["tag"] == "identifier":
        name = target["value"]
        destination = environment
        if name in ("__input", "__output"):
            destination = global_environment(environment)
        key = name
    elif target["tag"] == "complex":
        base, status = evaluate(target["base"], environment)
        if status is not None:
            return base, status
        index, status = evaluate(target["index"], environment)
        if status is not None:
            return index, status
        if type(base) is not list:
            raise TypeError("Indexed assignment requires a list")
        key = require_list_index(index, base)
        destination = base
    else:
        raise ValueError("Assignment requires an identifier or indexed destination")
    value, status = evaluate(ast["expression"], environment)
    if status is not None:
        return value, status
    destination[key] = value
```

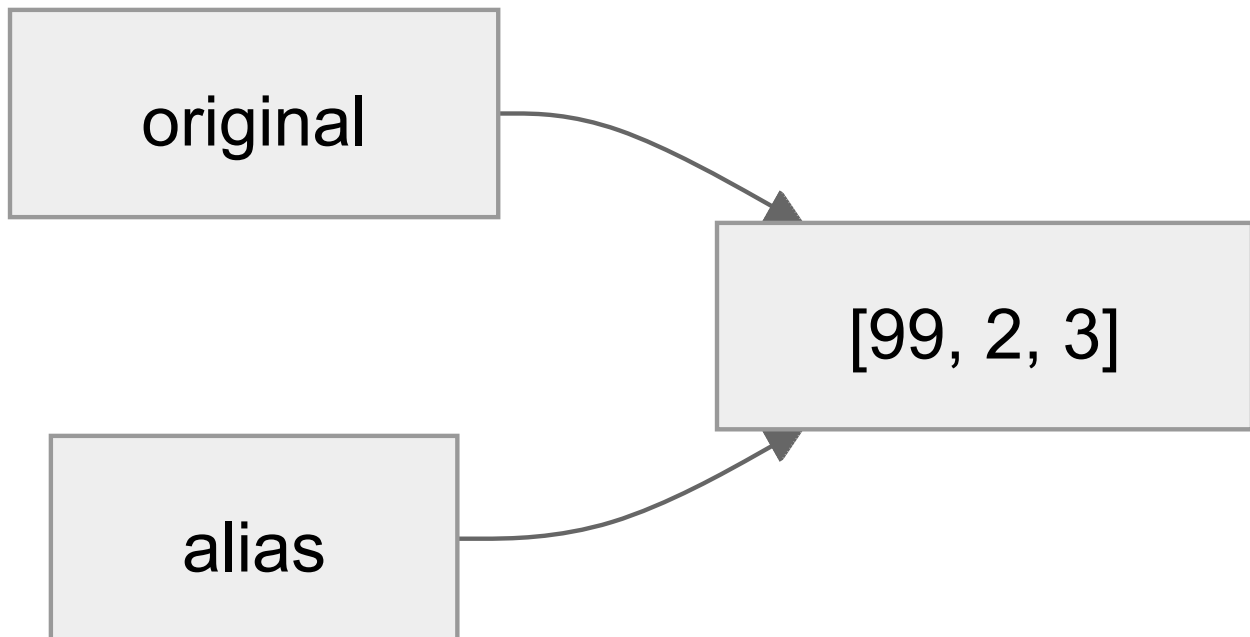
The order is deliberate, and worth stating plainly: **the target is resolved before the right-hand value is evaluated**, for every kind of target. For a plain identifier that resolution does nothing observable, so the order was never visible before this chapter. For an indexed target it is: `a[exit(6)] = exit(999)` returns `(6, "exit")`, not `(999, "exit")`, because the index is checked before the value is ever evaluated. If the base of an indexed target is itself an indexed read, as in a chained target, that read's own status is checked the same way, before the outer index and before the value. Nothing is stored, anywhere along the chain, once any of this returns a status: `a[0] = exit(9)`; leaves a byte-for-byte what it was.

Out-of-range assignment is rejected the same way an out-of-range read is: `a[5] = 1`; on a three-element list raises an error. It does not extend the list to make room.

Mutation and Rebinding

```
original = [1, 2, 3];  
alias = original;  
alias[0] = 99;  
print(original);
```

This prints `[99, 2, 3]`. `alias = original` did not copy the list; it copied the reference to it, the same thing every assignment in this interpreter has always done. Numbers and strings never exposed this, because they cannot be changed in place, only replaced. A list can be, so two names can now refer to the same list, and a mutation through either name is visible through both. No new code made this possible. The environment has stored plain references since chapter 2; a list is simply the first value where that fact becomes observable.

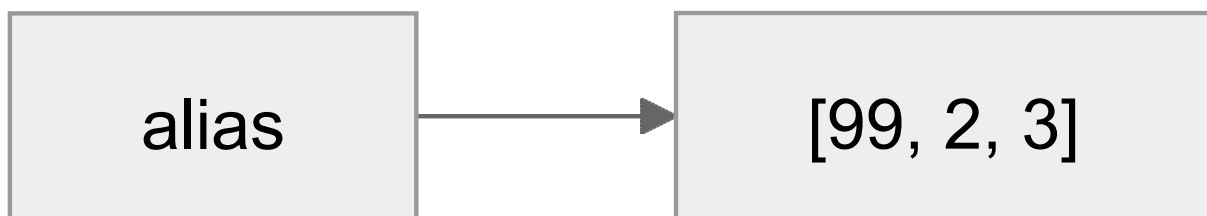
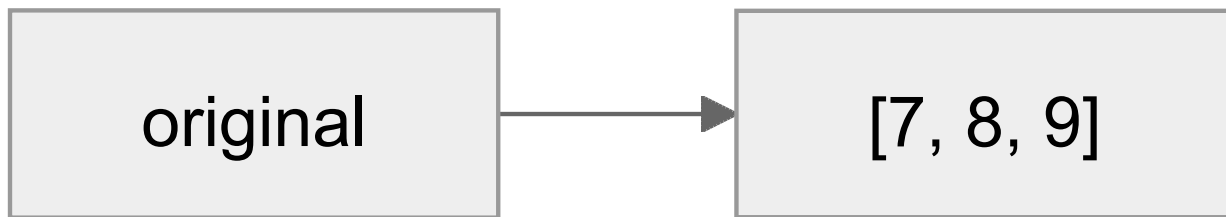


Both names point at the same list. There is only one list here, not two copies that happen to match.

Rebinding is different from mutating, and the two must not be confused:

```
original = [7, 8, 9];  
print(alias);
```

This still prints `[99, 2, 3]`. Assigning a new list to `original` gives that name a new list to point to. It says nothing about what `alias` points to, which has not changed.



original now points somewhere new. alias was never touched by that assignment and still points at the list it always did.

Equality

```
print([1, 2] == [1, 2]);  
print([true] == [1]);
```

This prints true, then false. Two lists are equal when they have the same length and their elements are equal pairwise, checked with this same rule recursively, not with Python's own ==:

```
def vertex_equal(left, right):  
    # Lists compare structurally, recursively applying this same rule to  
    # every element, rather than Python's own ==, which would let a  
    # nested true and 1 compare equal and break the Chapter 4 rule that  
    # they never do.  
    if type(left) is list and type(right) is list:  
        if len(left) != len(right):  
            return False  
        return all(vertex_equal(l, r) for l, r in zip(left, right))  
    numeric = type(left) in (int, float) and type(right) in (int, float)  
    return (numeric or type(left) is type(right)) and left == right
```

Python's own `[True] == [1]` is True, because Python considers True and 1 numerically equal everywhere, including inside a list. Vertex decided back in Chapter 4 that a boolean and a number are never equal, `true != 1`, and that rule has to keep holding no matter how deeply a value is nested inside a list for it to mean anything. `vertex_equal` is what makes that true recursively instead of only at the top level.

Ordering accepts two numbers or two strings, not lists. `[1] < [2]` is still a type error. The evaluator's operand check prevents Python's list ordering from becoming Vertex behavior by accident.

Concatenation

```
first = [1, 2];
second = [3, 4];
combined = first + second;
combined[0] = 99;
print(first);
```

This prints `[1, 2]`: `combined` is a new list, not a third name for `first`'s elements followed by `second`'s. `+` already overloaded for numbers and strings; extending it to two lists is the same kind of overload, and Python's own list `+` already builds a fresh list rather than sharing either operand, which is exactly the copy behavior concatenation needs here. No other arithmetic operator accepts a list on either side: `[1, 2] - [3]`, `[1] * 2`, and `[1, 2] + 3` are all type errors.

Printing and Conversions

```
print([1, "hi", true, [2, 3]]);
```

This prints `[1, "hi", true, [2, 3]]`. Printing a bare string still prints it unquoted, as it always has; a string sitting inside a list needs quotes, or `["hi"]` and `[hi]` would look the same with no way to tell an element that happens to be text apart from one that was never a string in the first place.

```
def vertex_repr(value):
    # The display form used inside a list, where a bare string needs
    # quotes to stay distinguishable from everything around it. print()
    # and string() use this for a list; a string or number printed on its
    # own still uses its plain, unquoted form.
    if type(value) is bool:
        return "true" if value else "false"
    if type(value) is list:
        return "[" + ", ".join(vertex_repr(item) for item in value) + "]"
    if type(value) is str:
        escaped = (value.replace("\\", "\\\").replace("'", '\\"'))
        .replace("\n", "\\n").replace("\t", "\\t").replace("\r", "\\r")
        return "'" + escaped + "'"
    return str(value)
```

`string()` uses the same representation for a list, so `string([1, 2])` is `"[1, 2]"`. `type([1, 2])` is `"list"`. `number()` and `boolean()` both reject a list outright: there is no sensible single number or truth value a list should become, so neither conversion pretends there is one.

Testing the Contract

`test_lists.py` covers empty and mixed-type literals; element evaluation order; nested literals and chained indexing; every rejected index case (fractional, boolean, string, negative, out of range); indexed assignment, including the out-of-range and non-list-base failures; mutation through an alias versus rebinding leaving the alias untouched; concatenation producing an independent list; structural equality, including the nested `true != 1` regression that Python's own list equality would miss; and `length`, `type`, `print`, and `string` for lists. `exit` is injected at every new position this chapter adds: a list element, an index base, an index, and an indexed assignment's right-hand value, checking both the propagated status and that nothing was mutated along the way.

Practice

1. Write a loop that finds the largest value in a list using only `while`, `length`, and indexing.
2. Update every element of a list in place by index, then compare the result through a second name that was assigned to the list beforehand. Confirm it sees the same changes.
3. Trace `grid[0][1] = exit(4);` on `grid = [[1, 2], [3, 4]]` by hand before running it. Which parts of the target are evaluated, and in what order, before the status is returned?

References

1. Wikipedia contributors. "Array (data type)." Wikipedia. [https://en.wikipedia.org/wiki/Array_\(data_type\)](https://en.wikipedia.org/wiki/Array_(data_type))

Questions for Thought

1. When should assigning a list copy it, and when should it share the same value? Which did this chapter choose, and why does that choice fall out of decisions made as early as Chapter 2?
2. How can the scalar equality rule `true != 1` affect the equality of two lists that look identical at a glance?
3. What state can change before an indexed assignment encounters `exit`, and what can only change after it?